SQL Practical: INSERT, UPDATE and DELETE

25 tasks with MySQL command-line code and illustrative outputs

How to run: Open MySQL Command Line Client, run `CREATE DATABASE CollegeDB;` and `USE CollegeDB;`, then run the table setup below. Execute each task in order. Outputs shown are representative MySQL results; exact affected-row counts depend on your existing data.

Initial setup (run once)

```
CREATE DATABASE IF NOT EXISTS CollegeDB;
USE CollegeDB;

CREATE TABLE IF NOT EXISTS Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT,
    Course VARCHAR(50),
    City VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS Employee (
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(50),
    Department VARCHAR(50),
    Salary DECIMAL(10,2)
);

CREATE TABLE IF NOT EXISTS Department (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(50),
    Location VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS Product (
    Product_ID INT PRIMARY KEY,
    Product_Name VARCHAR(50),
    Price DECIMAL(10,2),
    Quantity INT
);

CREATE TABLE IF NOT EXISTS Customer (
    Customer_ID INT PRIMARY KEY,
    Customer_Name VARCHAR(50),
    Email VARCHAR(100),
    Phone VARCHAR(15)
);

CREATE TABLE IF NOT EXISTS Course (
    Course_ID INT PRIMARY KEY,
    Course_Name VARCHAR(50),
    Duration INT,
    Fees DECIMAL(10,2)
);
```

Note: This setup uses Student_ID / Emp_ID / etc. as primary keys so duplicate IDs are rejected. If these tables already exist with different columns, adapt the INSERT statements to match your schema.

1. Insert one record into Student

```
INSERT INTO Student (Student_ID, Name, Age, Course, City)
VALUES (101, 'Amit', 20, 'BCA', 'Meerut');
```

Expected output:

```
Query OK, 1 row affected
```

2. Insert five student records using separate INSERT statements

```
INSERT INTO Student VALUES (102, 'Neha', 21, 'B.Tech', 'Delhi');
INSERT INTO Student VALUES (103, 'Rahul', 22, 'BCA', 'Meerut');
INSERT INTO Student VALUES (104, 'Priya', 20, 'BBA', 'Jaipur');
INSERT INTO Student VALUES (105, 'Karan', 23, 'B.Tech', 'Delhi');
INSERT INTO Student VALUES (106, 'Sneha', 19, 'BCA', 'Roorkee');
```

Expected output:

```
Each statement: Query OK, 1 row affected
```

3. Insert multiple records using one statement

```
INSERT INTO Student (Student_ID, Name, Age, Course, City)
VALUES
(107, 'Arjun', 21, 'BCA', 'Delhi'),
(108, 'Riya', 20, 'BBA', 'Meerut'),
(109, 'Vikas', 22, 'B.Tech', 'Noida');
```

Expected output:

```
Query OK, 3 rows affected
```

4. Insert records into Employee

```
INSERT INTO Employee VALUES
(201, 'Rohit', 'IT', 45000.00),
(202, 'Pooja', 'HR', 38000.00),
(203, 'Aman', 'Sales', 32000.00);
```

Expected output:

```
Query OK, 3 rows affected
```

5. Insert records into Department

```
INSERT INTO Department VALUES
(10, 'IT', 'Delhi'),
(20, 'HR', 'Meerut'),
(30, 'Sales', 'Noida');
```

Expected output:

```
Query OK, 3 rows affected
```

6. Insert records into Product

```
INSERT INTO Product VALUES
(301, 'Keyboard', 800.00, 15),
(302, 'Mouse', 450.00, 0),
(303, 'Monitor', 7500.00, 8);
```

Expected output:

```
Query OK, 3 rows affected
```

7. Insert records into Customer

```
INSERT INTO Customer VALUES
(401, 'Anjali Sharma', 'anjali@example.com', '9876500001'),
(402, 'Mohit Verma', 'mohit@example.com', '9876500002');
```

Expected output:

```
Query OK, 2 rows affected
```

8. Insert records into Course

```
INSERT INTO Course VALUES
(501, 'BCA', 36, 60000.00),
(502, 'B.Tech', 48, 120000.00),
(503, 'BBA', 36, 55000.00);
```

Expected output:

```
Query OK, 3 rows affected
```

9. Insert a student record with values for all columns

```
INSERT INTO Student (Student_ID, Name, Age, Course, City)
VALUES (110, 'Ishita', 21, 'BCA', 'Lucknow');
```

Expected output:

```
Query OK, 1 row affected
```

10. Insert a student by specifying selected columns

```
INSERT INTO Student (Student_ID, Name, City)
VALUES (111, 'Dev', 'Agra');
```

Expected output:

```
Query OK, 1 row affected
Age and Course are NULL because they were omitted.
```

11. Update the city of a particular student

```
UPDATE Student
SET City = 'Delhi'
WHERE Student_ID = 101;

SELECT Student_ID, Name, City
FROM Student WHERE Student_ID = 101;
```

Expected output:

```
Query OK, 1 row affected
+------------+------+-------+
| Student_ID | Name | City  |
+------------+------+-------+
| 101        | Amit | Delhi |
+------------+------+-------+
```

12. Update salary of a particular employee

```
UPDATE Employee
SET Salary = 50000.00
WHERE Emp_ID = 201;

SELECT Emp_ID, Emp_Name, Salary
FROM Employee WHERE Emp_ID = 201;
```

Expected output:

```
Query OK, 1 row affected
+--------+----------+----------+
| Emp_ID | Emp_Name | Salary   |
+--------+----------+----------+
| 201    | Rohit    | 50000.00 |
+--------+----------+----------+
```

13. Increase salary of all employees by 10%

```
UPDATE Employee
SET Salary = Salary * 1.10;

SELECT Emp_ID, Emp_Name, Salary FROM Employee;
```

Expected output:

```
Query OK, 3 rows affected
201 Rohit  55000.00
202 Pooja  41800.00
203 Aman   35200.00
```

14. Increase price of all products by 5%

```
UPDATE Product
SET Price = Price * 1.05;

SELECT Product_ID, Product_Name, Price FROM Product;
```

Expected output:

```
Query OK, 3 rows affected
301 Keyboard  840.00
302 Mouse     472.50
303 Monitor  7875.00
```

15. Change course of a particular student

```
UPDATE Student
SET Course = 'B.Tech'
WHERE Student_ID = 103;

SELECT Student_ID, Name, Course
FROM Student WHERE Student_ID = 103;
```

Expected output:

```
Query OK, 1 row affected
103 | Rahul | B.Tech
```

16. Update department of an employee

```
UPDATE Employee
SET Department = 'IT'
WHERE Emp_ID = 202;
```

```
SELECT Emp_ID, Emp_Name, Department
FROM Employee WHERE Emp_ID = 202;
```

Expected output:

```
Query OK, 1 row affected
202 | Pooja | IT
```

17. Update all students from Meerut to Delhi

```
UPDATE Student
SET City = 'Delhi'
WHERE City = 'Meerut';

SELECT Student_ID, Name, City
FROM Student WHERE City = 'Delhi';
```

Expected output:

```
Query OK, rows affected depends on matching records.
All students previously in Meerut now have City = Delhi.
```

18. Update fees of a particular course

```
UPDATE Course
SET Fees = 65000.00
WHERE Course_ID = 501;

SELECT Course_ID, Course_Name, Fees
FROM Course WHERE Course_ID = 501;
```

Expected output:

```
Query OK, 1 row affected
501 | BCA | 65000.00
```

19. Update multiple columns of a student simultaneously

```
UPDATE Student
SET Age = 22, Course = 'B.Tech', City = 'Jaipur'
WHERE Student_ID = 104;

SELECT Student_ID, Name, Age, Course, City
FROM Student WHERE Student_ID = 104;
```

Expected output:

```
Query OK, 1 row affected
104 | Priya | 22 | B.Tech | Jaipur
```

20. Delete a student using Student_ID

```
DELETE FROM Student
WHERE Student_ID = 111;

SELECT * FROM Student WHERE Student_ID = 111;
```

Expected output:

```
Query OK, 1 row affected
Empty set
```

21. Delete all students belonging to a particular city

```
DELETE FROM Student
WHERE City = 'Roorkee';

SELECT * FROM Student WHERE City = 'Roorkee';
```

Expected output:

```
Query OK, rows affected depends on matching records.
Empty set (if no Roorkee students remain).
```

22. Delete employees whose salary is below a specified amount

```
DELETE FROM Employee
WHERE Salary < 40000;

SELECT * FROM Employee WHERE Salary < 40000;
```

Expected output:

```
Query OK, rows affected depends on current salaries.
```

```
Empty set
```

23. Delete products whose quantity is zero

```
DELETE FROM Product
WHERE Quantity = 0;

SELECT * FROM Product WHERE Quantity = 0;
```

Expected output:

```
Query OK, 1 row affected (Mouse)
Empty set
```

24. Delete records satisfying multiple AND/OR conditions

```
-- AND: both conditions must be true
DELETE FROM Student
WHERE City = 'Delhi' AND Age < 20;

-- OR: either condition may be true
DELETE FROM Student
WHERE City = 'Agra' OR Course = 'BBA';

SELECT * FROM Student;
```

Expected output:

```
Query OK, affected-row count depends on remaining records.
Use SELECT before DELETE to preview matching rows.
```

25. Complete INSERT, UPDATE and DELETE sequence; display after each operation

```
-- Start with a clean, separate example ID
INSERT INTO Student (Student_ID, Name, Age, Course, City)
VALUES (900, 'Test Student', 20, 'BCA', 'Meerut');
SELECT * FROM Student WHERE Student_ID = 900;

UPDATE Student
SET City = 'Delhi', Age = 21
WHERE Student_ID = 900;
SELECT * FROM Student WHERE Student_ID = 900;

DELETE FROM Student
WHERE Student_ID = 900;
SELECT * FROM Student WHERE Student_ID = 900;
```

Expected output:

```
After INSERT:
900 | Test Student | 20 | BCA | Meerut

After UPDATE:
900 | Test Student | 21 | BCA | Delhi

After DELETE:
Empty set
```

Important practical notes

- Always use WHERE with UPDATE and DELETE when you intend to change only specific records.
- Before running a DELETE, use SELECT with the same WHERE clause to check which rows will be affected.
- Task 13 updates all current employee salaries by 10%. Do not run it repeatedly unless you want repeated increases.
- Task 14 updates all current product prices by 5%. Do not run it repeatedly unless you want repeated increases.
- If a task is rerun, primary-key duplicate errors may occur. Use different IDs or reset the practice tables carefully.
- Output row counts can differ based on existing data and MySQL settings; the displayed values are illustrative.